



# Stress Test Pipeline — Test Results

Automated Test Documentation for Model Governance

MKM Research Labs

Version 1.0 — 06-April-2026

David K Kelly

**CONFIDENTIAL — PROPRIETARY**

## **Legal Notice**

### **PROPRIETARY AND CONFIDENTIAL**

This document, including all algorithms, models, methodology, formulas, calculations, and intellectual concepts contained herein, constitutes the exclusive intellectual property and confidential trade secrets of MKM Research Labs (“MKM”).

Any usage, reproduction, distribution, modification, or disclosure of this document or any portion thereof, without the express prior written authorisation from MKM Research Labs is strictly prohibited and constitutes an infringement of intellectual property rights.

The document is provided on an “as is” basis. MKM Research Labs makes no representations or warranties of any kind, express or implied, regarding its accuracy, reliability, or suitability for any particular purpose.

All rights reserved. © 2021–2026 MKM Research Labs.

Governed by the laws of the United Kingdom.

# 1 Test Results — Stress Test Pipeline (MKM-ST-001)

Tests run on **06 April 2026 at 07:49:50**.

Table 1: Test Results Summary — Stress Test Pipeline (MKM-ST-001)

|             |        |
|-------------|--------|
| Total Tests | 107    |
| Passed      | 107    |
| Failed      | 0      |
| Skipped     | 0      |
| Duration    | 20.75s |

Table 2: Detailed Test Results — Stress Test Pipeline

| Test Description                                           | Acceptance Criteria                                                                           | Result | Time   |
|------------------------------------------------------------|-----------------------------------------------------------------------------------------------|--------|--------|
| All already trained                                        | <code>result["trained"] == 0;</code><br><code>result["skipped"] == 3</code>                   | Pass   | 0.002s |
| After the first gauge trains, ETA should appear in output. | <code>"ETA" in captured</code>                                                                | Pass   | 0.220s |
| Missing gauge json                                         | —                                                                                             | Pass   | 0.000s |
| Missing sequences                                          | —                                                                                             | Pass   | 0.001s |
| No valid gauges                                            | <code>result["trained"] == 0</code>                                                           | Pass   | 0.001s |
| One gauge already trained, one succeeds, one fails.        | <code>result["trained"] == 1;</code><br><code>result["failed"] == 1 (+1 more)</code>          | Pass   | 0.156s |
| Successful training                                        | <code>result["trained"] == 3;</code><br><code>result["failed"] == 0 (+2 more)</code>          | Pass   | 0.229s |
| Training failure is caught                                 | <code>result["trained"] == 0;</code><br><code>result["failed"] == 3 (+1 more)</code>          | Pass   | 0.243s |
| Counts default to zero                                     | <code>s["alert_sequences"] == 0;</code><br><code>s["warning_sequences"] == 0</code> (+1 more) | Pass   | 0.001s |
| Num gauges                                                 | <code>s["num_gauges"] == 4</code>                                                             | Pass   | 0.001s |
| Num sequences                                              | <code>s["num_sequences"] == 7</code>                                                          | Pass   | 0.001s |
| Returns required keys                                      | <code>key in s</code>                                                                         | Pass   | 0.001s |
| When one gauge's classifier fails, pipeline continues.     | <code>"classifier failed" in out;</code><br><code>result["num_gauges"] == 2</code>            | Pass   | 0.467s |
| Dict format                                                | <code>len(result) == 2</code>                                                                 | Pass   | 0.000s |
| Empty list                                                 | <code>_extract_gauges({"flood_gauges": []}) == []</code>                                      | Pass   | 0.000s |
| List format                                                | <code>result == [_G1, _G2]</code>                                                             | Pass   | 0.000s |
| Missing key returns empty                                  | <code>_extract_gauges({}) == []</code>                                                        | Pass   | 0.000s |

| Test Description                                                      | Acceptance Criteria                                                        | Result | Time   |
|-----------------------------------------------------------------------|----------------------------------------------------------------------------|--------|--------|
| When gaugehd/ has baseline data, the 'Loaded' message appears.        | "gauge baselines from gaugehd" in out or "heuristic base ...               | Pass   | 0.479s |
| Gauge summary dir written                                             | sg_dir.is_dir()                                                            | Pass   | 0.000s |
| Gauge summary gauge ids                                               | len(d["gauge_ids"]) == 3                                                   | Pass   | 0.000s |
| Gauge summary num gauges                                              | d["num-gauges"] == 3                                                       | Pass   | 0.000s |
| Gauge summary schema version                                          | meta.exists(); d["schema_version"] == SCHEMA_VERSION_SPATIAL               | Pass   | 0.001s |
| Gauge summary sequence count                                          | len(gauge_files) == 3                                                      | Pass   | 0.000s |
| Sequences file loadable                                               | len(seqs) == 40                                                            | Pass   | 0.001s |
| Sequences json written                                                | (gauge_dir / SEQUENCES_FILENAME).exists()                                  | Pass   | 0.000s |
| Summary json written                                                  | (gauge_dir / SUMMARY_FILENAME).exists()                                    | Pass   | 0.000s |
| Alert count non negative                                              | full_run["alert_sequences"] >= 0                                           | Pass   | 0.000s |
| Elapsed positive                                                      | full_run["elapsed_seconds"] > 0                                            | Pass   | 0.000s |
| Num gauges                                                            | full_run["num-gauges"] == 3                                                | Pass   | 0.000s |
| Num sequences                                                         | full_run["num-sequences"] == 40                                            | Pass   | 0.000s |
| Returns dict                                                          | isinstance(full_run, dict)                                                 | Pass   | 0.000s |
| Severe lte warning                                                    | full_run["severe_sequences"] <= full_run["warning_sequenc...               | Pass   | 0.000s |
| Type counts present                                                   | "type_counts" in full_run; sum(full_run["type_counts"].values()) == 40     | Pass   | 0.000s |
| Warning lte alert                                                     | full_run["warning_sequences"] <= full_run["alert_sequences"]               | Pass   | 0.000s |
| Batch generator populates category                                    | all(s.intensity_category in valid for s in seqs)                           | Pass   | 0.001s |
| From dict missing category defaults empty                             | reconstructed.intensity_category == ""                                     | Pass   | 0.000s |
| From dict round trip                                                  | reconstructed.intensity_category == s.intensity_category                   | Pass   | 0.001s |
| Serialized sequences have category                                    | all(s.intensity_category in valid for s in seqs)                           | Pass   | 0.001s |
| To dict includes intensity category                                   | "intensity_category" in d; d["intensity_category"] == s.intensity_category | Pass   | 0.001s |
| Legacy sequence_gauge_summary.json is deleted after split-mode write. | not legacy.exists()                                                        | Pass   | 0.469s |
| Corrupt file skipped                                                  | len(baselines) == 1; "GAUGE-OK" in baselines                               | Pass   | 0.002s |

| Test Description                                                      | Acceptance Criteria                                                                                           | Result | Time   |
|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|--------|--------|
| Empty directory                                                       | <code>_load_gaugehd.baselines(d) == {}</code>                                                                 | Pass   | 0.000s |
| Missing directory                                                     | <code>_load_gaugehd.baselines(tmp_path / "nonexistent") == {}</code>                                          | Pass   | 0.000s |
| Missing mean level skipped                                            | <code>len(baselines) == 0</code>                                                                              | Pass   | 0.002s |
| Multiple gauges                                                       | <code>len(baselines) == 3</code>                                                                              | Pass   | 0.003s |
| Valid gaugehd files                                                   | <code>"GAUGE-001" in baselines;</code><br><code>baselines["GAUGE-001"]["mean_level"] == 1.25 (+1 more)</code> | Pass   | 0.001s |
| Sequence with malformed sequence_start doesn't crash.                 | <code>result["num_sequences"] == 5</code>                                                                     | Pass   | 0.464s |
| Explicitly test malformed date by patching sequences post-generation. | <code>result["num_sequences"] == 3</code>                                                                     | Pass   | 0.468s |
| No crash when gauge json absent                                       | <code>isinstance(result, dict)</code>                                                                         | Pass   | 0.002s |
| Num gauges zero without gauge json                                    | <code>result["num_gauges"] == 0</code>                                                                        | Pass   | 0.001s |
| Sequences still written without gauge json                            | <code>(tmp_path / SEQUENCES_FILENAME).exists()</code>                                                         | Pass   | 0.002s |
| gauge.json present but every record missing gauge_id → n_all == 0.    | <code>isinstance(result, dict);</code><br><code>result["num_gauges"] == 0 (+1 more)</code>                    | Pass   | 0.002s |
| Sequences still written when all gauges invalid                       | <code>(tmp_path / SEQUENCES_FILENAME).exists()</code>                                                         | Pass   | 0.002s |
| count=2001 with verbose=False prints ETA line at i=2000.              | <code>result["num_sequences"] == 2001;</code><br><code>"2,000" in out or "ETA" in out</code>                  | Pass   | 2.596s |
| Base level derived from alert                                         | <code>r["base_level"] == pytest.approx(3.5 * 0.35)</code>                                                     | Pass   | 0.000s |
| Flood alert                                                           | <code>_parse_gauge(_G1)["flood_alert"] ==</code><br><code>pytest.approx(3.5)</code>                           | Pass   | 0.000s |
| Flood warning                                                         | <code>_parse_gauge(_G1)["flood_warning"] ==</code><br><code>pytest.approx(4.6)</code>                         | Pass   | 0.000s |
| Gauge id                                                              | <code>_parse_gauge(_G1)["gauge_id"] ==</code><br><code>"GAUGE-test001"</code>                                 | Pass   | 0.000s |
| Lat lon                                                               | <code>r["lat"] == pytest.approx(51.46);</code><br><code>r["lon"] == pytest.approx(-0.30)</code>               | Pass   | 0.000s |
| Missing alert returns none                                            | <code>_parse_gauge(bad) is None</code>                                                                        | Pass   | 0.000s |
| Missing gauge id returns none                                         | <code>_parse_gauge(bad) is None</code>                                                                        | Pass   | 0.000s |
| Missing lat returns none                                              | <code>_parse_gauge(bad) is None</code>                                                                        | Pass   | 0.000s |
| Severe warning                                                        | <code>_parse_gauge(_G1)["severe_warning"]</code><br><code>== pytest.approx(5.5)</code>                        | Pass   | 0.000s |
| Valid record returns dict                                             | <code>result is not None</code>                                                                               | Pass   | 0.000s |
| Warning defaults when absent                                          | <code>r is not None; r["flood_warning"] ==</code><br><code>pytest.approx(4.0 * 1.33) (+1 more)</code>         | Pass   | 0.000s |

| Test Description                                                    | Acceptance Criteria                                                                                                                                                                     | Result | Time   |
|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|--------|
| Baseline without monthly means                                      | <code>result["base_level"] == 1.5;</code>                                                                                                                                               | Pass   | 0.000s |
| With baseline                                                       | <code>result["monthly_means"] is None</code><br><code>result["base_level"] == 1.25;</code><br><code>result["monthly_means"] == {"01": 1.4}</code>                                       | Pass   | 0.000s |
| Without baseline uses heuristic                                     | <code>result["base_level"] ==</code><br><code>pytest.approx(3.5 * 0.35);</code><br><code>result["monthly_means"] is None</code>                                                         | Pass   | 0.000s |
| Empty monthly means                                                 | <code>_seasonal_base_level({},</code><br><code>mean_level=1.0, month=6) == 1.0</code>                                                                                                   | Pass   | 0.000s |
| Missing month falls back                                            | <code>_seasonal_base_level(monthly,</code><br><code>mean_level=1.0, month=3) ==...</code>                                                                                               | Pass   | 0.000s |
| No monthly means                                                    | <code>_seasonal_base_level(None,</code><br><code>mean_level=1.0, month=6) == 1.0</code>                                                                                                 | Pass   | 0.000s |
| With monthly means                                                  | <code>_seasonal_base_level(monthly,</code><br><code>mean_level=1.0, month=1) ==...;</code><br><code>_seasonal_base_level(monthly,</code><br><code>mean_level=1.0, month=7) ==...</code> | Pass   | 0.000s |
| All have intensity category                                         | <code>all(r["intensity_category"] in valid</code><br><code>for r in records)</code>                                                                                                     | Pass   | 0.000s |
| All have sequence id                                                | <code>all("sequence_id" in r for r in</code><br><code>records)</code>                                                                                                                   | Pass   | 0.000s |
| All have sequence type                                              | <code>all(r["sequence_type"] in valid for r</code><br><code>in records)</code>                                                                                                          | Pass   | 0.000s |
| All have total precip                                               | <code>all("total_precip_mm" in r for r in</code><br><code>records); all(r["total_precip_mm"] &gt; 0</code><br><code>for r in records)</code>                                            | Pass   | 0.000s |
| Gauge file count matches num gauges                                 | <code>len(gauge_records) == 3</code>                                                                                                                                                    | Pass   | 0.000s |
| Peak hours in valid range                                           | <code>0 &lt;= s["peak_hour"] &lt;= 167</code>                                                                                                                                           | Pass   | 0.000s |
| Peaks positive                                                      | <code>all(s["peak_m"] &gt; 0 for s in seqs)</code>                                                                                                                                      | Pass   | 0.000s |
| Per gauge records have alert flags                                  | <code>all("alert" in s for s in seqs)</code>                                                                                                                                            | Pass   | 0.000s |
| Per gauge records have peak                                         | <code>all("peak_m" in s for s in seqs)</code>                                                                                                                                           | Pass   | 0.000s |
| Per gauge records have peak hour                                    | <code>all("peak_hour" in s for s in seqs)</code>                                                                                                                                        | Pass   | 0.000s |
| Per gauge sequence count                                            | <code>len(seqs) == len(records)</code>                                                                                                                                                  | Pass   | 0.000s |
| Severe implies warning                                              | <code>s["warning"]</code>                                                                                                                                                               | Pass   | 0.000s |
| Every sequence that breached warning must also have breached alert. | <code>s["alert"]</code>                                                                                                                                                                 | Pass   | 0.000s |
| Error message lists available ids                                   | <code>"GAUGE-test001" in</code><br><code>str(exc_info.value)</code>                                                                                                                     | Pass   | 0.001s |
| Invalid gauge id raises                                             | —                                                                                                                                                                                       | Pass   | 0.001s |

| Test Description                                                | Acceptance Criteria                                                                                                                     | Result | Time   |
|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|--------|--------|
| Named file gauge ids                                            | <code>d["gauge_ids"] == ["GAUGE-test002"]</code>                                                                                        | Pass   | 0.000s |
| Named file peaks length one                                     | <code>all(len(r["peaks_m"]) == 1 for r in d["sequences"])</code>                                                                        | Pass   | 0.000s |
| Named file schema version                                       | <code>d["schema_version"] == SCHEMA_VERSION_SPATIAL</code>                                                                              | Pass   | 0.003s |
| Named file sequence count                                       | <code>len(d["sequences"]) == 40</code>                                                                                                  | Pass   | 0.001s |
| Named output file written                                       | <code>(gauge_dir / "sequence_gauge_GAUGE-test002.json").exists()</code>                                                                 | Pass   | 0.000s |
| Summary num gauges is one                                       | <code>single_run["num_gauges"] == 1</code>                                                                                              | Pass   | 0.000s |
| Single gauge classifier result dict                             | <code>isinstance(result, dict)</code>                                                                                                   | Pass   | 6.168s |
| Single gauge + train_classifier=True covers lines 353-361.      | <code>result["num_gauges"] == 1; stressm_dir.exists()</code>                                                                            | Pass   | 6.086s |
| Base level matches gauge params                                 | <code>responses["GAUGE-A"][0]["base_level_m"] == pytest.approx(...; responses["GAUGE-B"][0]["base_level_m"] == pytest.approx(...</code> | Pass   | 0.000s |
| Gauge a values                                                  | <code>r["base_level_m"] == pytest.approx(1.5); r["peak_level_m"] == pytest.approx(5.0) (+1 more)</code>                                 | Pass   | 0.000s |
| Gauge b values                                                  | <code>r["base_level_m"] == pytest.approx(2.0); r["peak_level_m"] == pytest.approx(7.0) (+1 more)</code>                                 | Pass   | 0.000s |
| Level change is peak minus base                                 | <code>r["level_change_m"] == pytest.approx(expected)</code>                                                                             | Pass   | 0.000s |
| Response has base level m                                       | <code>"base_level_m" in r</code>                                                                                                        | Pass   | 0.000s |
| Response has level change m                                     | <code>"level_change_m" in r</code>                                                                                                      | Pass   | 0.000s |
| Creates new summary                                             | <code>summary["num_gauges"] == 1; summary["num_trained"] == 1 (+1 more)</code>                                                          | Pass   | 0.003s |
| Merges into existing                                            | <code>summary["num_gauges"] == 2; summary["num_trained"] == 2 (+1 more)</code>                                                          | Pass   | 0.002s |
| Replaces existing gauge                                         | <code>summary["num_gauges"] == 1; summary["gauges"][0]["metrics"]["auc_roc"] == 0.95</code>                                             | Pass   | 0.002s |
| Skipped gauge not counted as trained                            | <code>summary["num_trained"] == 0; summary["num_skipped"] == 1</code>                                                                   | Pass   | 0.001s |
| With count ; 2000 and verbose=False, no mid-loop progress line. | <code>"ETA" not in out or "[" not in out</code>                                                                                         | Pass   | 1.076s |

| Test Description                                                 | Acceptance Criteria                                            | Result | Time   |
|------------------------------------------------------------------|----------------------------------------------------------------|--------|--------|
| count=501 with verbose=True triggers the progress line at i=500. | result["num_sequences"] == 501; "[500" in out or "500/" in out | Pass   | 1.577s |

1.1 Test Document History

| Date        | Version | Description                     | Author   |
|-------------|---------|---------------------------------|----------|
| 27-Mar-2026 | 1.0     | Initial test suite (126 tests)  | D. Kelly |
| 06-Apr-2026 | 1.1     | 19 test(s) removed (total: 107) | D. Kelly |